

Actividad Integradora Lombok-DTO Programación III

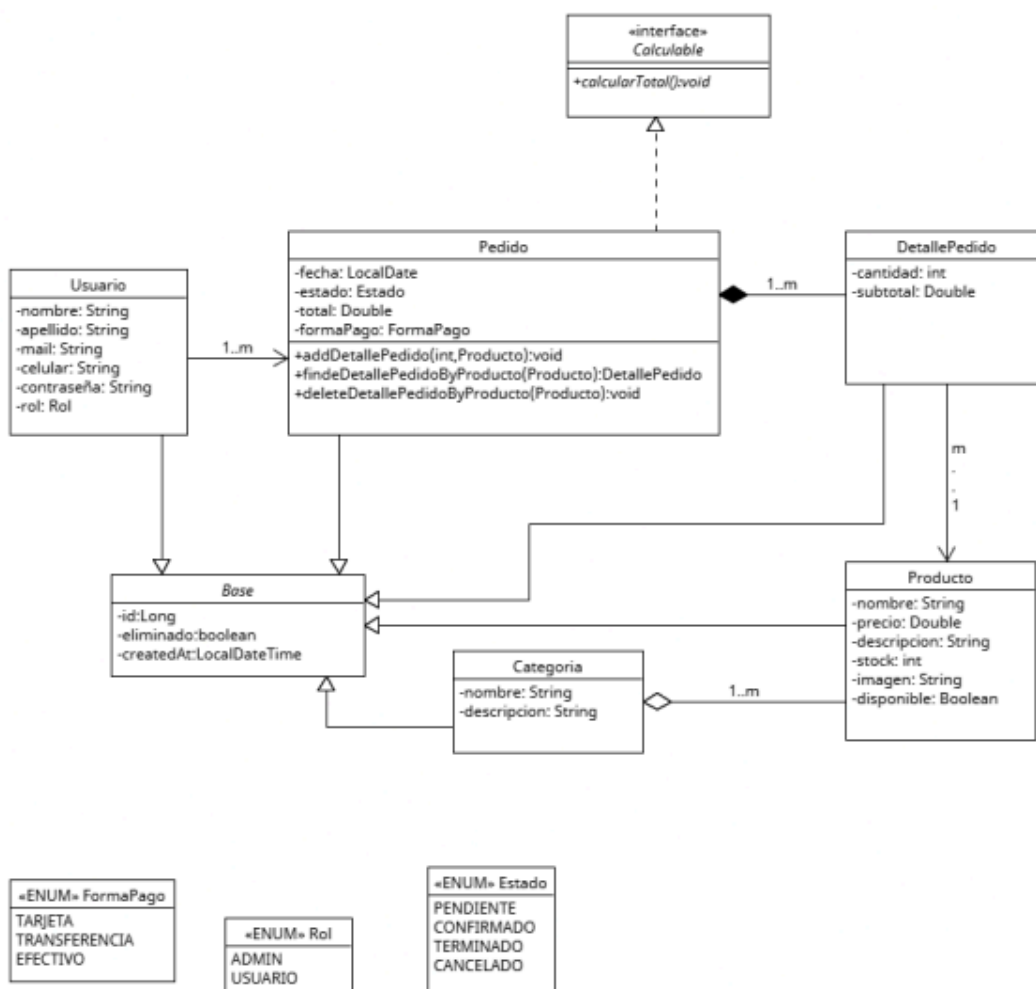
Coordinador: Enferrel, Ariel
Profesor/es: Rigoni, Cinthia
Tutor/a:
Alumno/a: Lauk, Karen

Objetivo

Hacer uso de librería lombok para simplificar código repetitivo, crear DTOs para ocultar información sensible.

Caso Práctico

UML



Consignas

1. Para el desarrollo de este trabajo práctico se deberá tomar como base el modelo de clases desarrollado en la Unidad 5. A diferencia del trabajo anterior, en esta instancia el proyecto deberá configurarse y desarrollarse utilizando Gradle como herramienta de gestión de dependencias y construcción. Asimismo, se deberá incorporar la librería Lombok, la cual será utilizada para reemplazar código repetitivo mediante anotaciones.

Se configuró el proyecto utilizando Gradle. En el archivo build.gradle se añadieron las dependencias necesarias para habilitar las anotaciones de Lombok en tiempo de compilación y ejecución

2. Insertar anotaciones Lombok, se deberán utilizar al menos las siguientes anotaciones:

Librería Lombok → herramienta utilizada para reducir el código repetitivo que no aporta lógica de negocio

Se utilizaron las siguientes anotaciones obligatorias:

a. Getter/Setter

Permiten acceder y modificar atributos privados sin implementar manualmente los métodos getter y setter.

b. ToString

Genera automáticamente una representación textual del objeto para facilitar su visualización por consola.

c. EqualsAndHashCode

Fundamental para las colecciones que no permiten duplicados (Set). En clases con herencia, se configuró con callSuper = false para aislar las propiedades de la clase hija y evitar referencias circulares o errores de desborde de memoria.

- @ToString(callSuper = true)

La configuración callSuper = true permite incluir también los atributos heredados de la clase padre (Base), como por ejemplo id, createdAt o cualquier otro dato común a todas las entidades.

- @EqualsAndHashCode(callSuper = false, of = "nombre")

Se configuró para comparar los objetos utilizando únicamente el atributo de negocio nombre, evitando depender del identificador heredado. Esto resulta fundamental para detectar productos duplicados dentro de colecciones de tipo Set.

d. Builder / SuperBuilder

Implementan el patrón Builder, permitiendo construir objetos complejos mediante una sintaxis fluida y legible.

e. AllArgsConstructor

Genera automáticamente un constructor con todos los atributos.

f. NoArgsConstructor

Genera un constructor vacío, útil para frameworks y procesos de serialización o mapeo de objetos.

3. En el método main se deberán instanciar utilizando patron builder

a. 2 Usuarios

- adminKaren
- clientejuan.

b. 3 Pedidos (al menos 2 detalles pedido por cada uno)

Instanciaste:

- pedidoFinde
- pedidoCena
- pedidoAlmuerzo

Cada pedido contiene al menos dos detalles de pedido.

c. 3 Categorías

d. 10 productos

Cargue las categorías y 10 Productos pertenecientes al catálogo de FoodStore

4. En la clase Main se deberán tener las instancias solicitadas en el punto anterior y se deberá utilizar el método toString para mostrar por consola un producto, el listado de productos cargados y los pedidos del usuario que posea la mayor cantidad de pedidos.

Se utilizó el método toString() generado automáticamente por Lombok para mostrar información por consola.

Las pruebas realizadas fueron:

1. Producto individual
2. Listado completo de productos
3. Pedidos del usuario con mayor actividad

5. Instanciar un nuevo producto donde el/los campos utilizados en el método equals sean iguales a los de otro producto existente. Luego, comparar dicha instancia con todos los elementos de la colección de productos y mostrar los resultados por pantalla.

Para almacenar los productos se utilizó una colección de tipo Set.

→ Set<Producto> catalogo = new HashSet<>();

A diferencia de una lista (List), un Set no permite elementos duplicados.

→ @EqualsAndHashCode(of = "nombre")

dos productos con el mismo nombre son considerados equivalentes.

Resultado obtenido:

¿El catálogo detecta que ya existe?: true

¿Se permitió ingresar el duplicado?: false

Cantidad de ítems remanentes: 10

Esto demuestra que la colección rechazó automáticamente la inserción de un producto repetido, preservando la integridad del catálogo.

6. Crear un nuevo paquete llamado DTOs y, dentro de él, una clase record llamada UsuarioDTO, que contendrá la misma información que la clase Usuario, evitando mostrar información sensible. Se deberán ocultar los siguientes atributos:
 - a. Rol
 - b. Contraseña

Se creó el paquete: com.utn.dtos

Dentro del mismo se implementó la clase:

```
public record UsuarioDTO(
    String nombre,
    String apellido,
    String mail
) {
    public static UsuarioDTO desdeEntidad(Usuario usuario) {
        return new UsuarioDTO(
            usuario.getNombre(),
            usuario.getApellido(),
            usuario.getMail()
        );
    }
}
```

Se utilizó un record porque proporciona una estructura inmutable, concisa y orientada al transporte de datos.

Además, se ocultó información sensible de la entidad Usuario, específicamente:

- Rol
- Contraseña

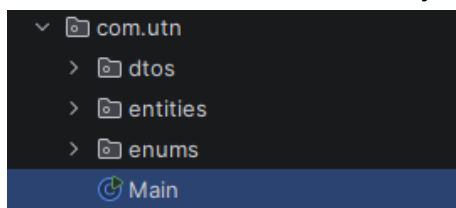
De esta forma, solamente se exponen los datos necesarios para la visualización.

Resultado obtenido:

UsuarioDTO[nombre=Karen, apellido=Lauk, mail=karen.lauk@mail.com]

Estructura y arquitectura

1. com.utn: Contiene la clase de ejecución Main



Main:

```
1 package com.utn;
2 import com.utn.dtos.UsuarioDTO;
3 import com.utn.entities.*;
4 import com.utn.enums.*;
5 import java.time.LocalDate;
6 import java.util.HashSet;
7 import java.util.Set;
8
9 public class Main {
10     public static void main(String[] args) {
11
12         // INSTANCIAR LAS CATEGORÍAS REALES DEL FRONTEND
13         Categoria catBurgers = Categoria.builder().nombre("Hamburguesas").descripcion("Variedad de hamburguesas smash").build();
14         Categoria catPizzas = Categoria.builder().nombre("Pizzas").descripcion("Pizzas artesanales al horno").build();
15         Categoria catBebidas = Categoria.builder().nombre("Bebidas").descripcion("Bebidas y gaseosas líneas premium").build();
16         Categoria catGuarniciones = Categoria.builder().nombre("Guarniciones").descripcion("Acompañamientos crujientes").build();
17         Categoria catEnsaladas = Categoria.builder().nombre("Ensaladas").descripcion("Opciones frescas y saludables").build();
18         Categoria catSandwiches = Categoria.builder().nombre("Sandwiches").descripcion("Sandwiches gourmet en pan baguette").build();
19
20         // INSTANCIAR LOS PRODUCTOS REALES (Mismos nombres y precios que el archivo de TypeScript)
21         Set<Producto> menuProductos = new HashSet<>();
22
23         // Hamburguesas
24         Producto burgerTriple = Producto.builder().nombre("Hamburguesa Triple").precio(25000.0).categoria(catBurgers).build();
25         Producto burgerSmash = Producto.builder().nombre("Hamburguesa Smash Supremo").precio(22000.0).categoria(catBurgers).build();
26
27         // Pizzas
28         Producto pizzaCalabresa = Producto.builder().nombre("Pizza Calabresa").precio(20500.0).categoria(catPizzas).build();
29         Producto pizzaMozzarella = Producto.builder().nombre("Pizza Mozzarella").precio(18500.0).categoria(catPizzas).build();
30
31         // Bebidas
32         Producto bebidaCola = Producto.builder().nombre("Bebida Cola").precio(1500.0).categoria(catBebidas).build();
33     }
34 }
```

```
// Guarniciones
Producto papasFritas = Producto.builder().nombre("Papas Fritas").precio(7500.0).categoria(catGuarniciones).build();
Producto papasCheddar = Producto.builder().nombre("Papas Fritas con Cheddar").precio(9000.0).categoria(catGuarniciones).build();
```

```
// Ensaladas y Sandwiches
Producto ensaladaGarden = Producto.builder().nombre("Garden Ensalada").precio(15500.0).categoria(catEnsaladas).build();
Producto sandwichItaliano = Producto.builder().nombre("Sandwich Italiano").precio(17000.0).categoria(catSandwiches).build();
Producto sandwichMilanesas = Producto.builder().nombre("Sandwich Milanesa").precio(18000.0).categoria(catSandwiches).build();
```

```
// Agregamos los 10 productos reales al catálogo
```

```
menuProductos.add(burgerTriple);
menuProductos.add(burgerSmash);
menuProductos.add(pizzaCalabresa);
menuProductos.add(pizzaMuzzarella);
menuProductos.add(bebidaCola);
menuProductos.add(papasFritas);
menuProductos.add(papasCheddar);
menuProductos.add(ensaladaGarden);
menuProductos.add(sandwichItaliano);
menuProductos.add(sandwichMilanesas);
```

```
// INSTANCIAR 3 PEDIDOS REALIZADOS EN EL LOCAL
```

```
// Pedido 1: Combo de hamburguesas para el fin de semana
```

```
Pedido pedidoFinde = Pedido.builder().fecha(LocalDate.now()).estado(Estado.PENDIENTE).formaPago(FormaPago.EFECTIVO).build();
pedidoFinde.addDetallePedido( cantidad: 2, burgerTriple); // 2 Hamburguesas Triples
pedidoFinde.addDetallePedido( cantidad: 2, bebidaCola); // 2 Bebidas Cola
pedidoFinde.calcularTotal(); // Orden correcto: Calcula e impacta internamente mediante el setTotal de Lombok
```

```
// Pedido 2: Cena de pizzas y guarnición
```

```
Pedido pedidoCena = Pedido.builder().fecha(LocalDate.now()).estado(Estado.CONFIRMADO).formaPago(FormaPago.TARJETA).build();
pedidoCena.addDetallePedido( cantidad: 1, pizzaCalabresa); // 1 Pizza Calabresa
pedidoCena.addDetallePedido( cantidad: 1, papasCheddar); // 1 Papas con Cheddar
pedidoCena.calcularTotal(); // Orden correcto: Calcula e impacta internamente mediante el setTotal de Lombok
```

```
// Pedido 3: Opción más liviana o almuerzo rápido
```

```
Pedido pedidoAlmuerzo = Pedido.builder().fecha(LocalDate.now()).estado(Estado.PENDIENTE).formaPago(FormaPago.TRANSFERENCIA).build();
pedidoAlmuerzo.addDetallePedido( cantidad: 1, sandwichItaliano); // 1 Sandwich Italiano
pedidoAlmuerzo.addDetallePedido( cantidad: 1, ensaladaGarden); // 1 Garden Ensalada
pedidoAlmuerzo.calcularTotal(); // Orden correcto: Calcula e impacta internamente mediante el setTotal de Lombok
```

```
// INSTANCIAR 2 USUARIOS REALES (Admin Karen Lauk y un Cliente de prueba)
```

```
Usuario adminKaren = Usuario.builder().nombre("Karen").apellido("Lauk").mail("karen.lauk@mail.com").contraseña("KarenPass2026").rol(Rol.ADMIN).build();
Usuario clientejuan = Usuario.builder().nombre("Juan").apellido("Lopez").mail("Juan@gmail.com").contraseña("JuanLopez").rol(Rol.USUARIO).build();
```

```
// Vinculamos los pedidos calculados a los historiales de los usuarios
```

```
adminKaren.getPedidos().add(pedidoFinde);
adminKaren.getPedidos().add(pedidoCena);
clientejuan.getPedidos().add(pedidoAlmuerzo);
```

```
Set<Usuario> listaUsuarios = Set.of(adminKaren, clientejuan);
```

```
// MUESTRAS SOLICITADAS POR CONSOLA
```

```
System.out.println("VISTA DE UN PRODUCTO INDIVIDUAL:");
System.out.println(burgerTriple);
```

```
System.out.println("\nMENU COMPLETO DISPONIBLE EN EL FOOD STORE:");
```

```
for (Producto prod : menuProductos) {
    System.out.println(prod);
}
```

```
// BUSQUEDA MANUAL DEL USUARIO CON MAS PEDIDOS (Estilo Algoritmico Tradicional)
System.out.println("\nDETECCION DEL CLIENTE CON MAS ACTIVIDAD:");
Usuario usuarioMasActivo = null;
int maxPedidos = -1;

for (Usuario user : listaUsuarios) {
    if (user.getPedidos().size() > maxPedidos) {
        maxPedidos = user.getPedidos().size();
        usuarioMasActivo = user;
    }
}

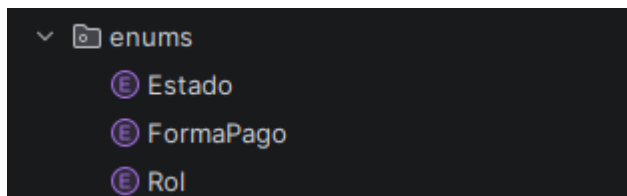
if (usuarioMasActivo != null) {
    System.out.println("Cliente frecuente detectado: " + usuarioMasActivo.getNombre() + " " + usuarioMasActivo.getApellido());
    System.out.println("Historial de compras de este usuario:");
    for (Pedido ped : usuarioMasActivo.getPedidos()) {
        System.out.println(ped); // Imprime toda la información estructurada por Lombok
        System.out.println("-> Total leído desde el atributo del Pedido: $" + ped.getTotal()); // Muestra el total real calculado
    }
}

// VALIDACIÓN DE EQUALS Y HASHCODE (Control de Duplicados en la Carta)
System.out.println("\nDETECCION DE COMIDA DUPLICADA EN LA CARTA:");
// Intentamos crear un producto lógicamente igual a uno existente (Mismo nombre)
Producto hamburguesaClonada = Producto.builder() ProductoBuilder<capture of ?, capture of ?>
    .nombre("Hamburguesa Triple") capture of ?
    .precio(25000.0)
    .categoria(catBurgers)
    .build();

System.out.println("El catalogo detecta que la hamburguesa ya existe logicamente?: " + menuProductos.contains(hamburguesaClonada));
boolean sePudoAgregar = menuProductos.add(hamburguesaClonada);
System.out.println("El sistema permitio ingresar el elemento duplicado?: " + sePudoAgregar);
System.out.println("Cantidad de items reales remanentes en el menu: " + menuProductos.size());
```

```
// MOSTRAR LA VISTA OBLIGATORIA DEL DTO
System.out.println("\nVISTA DTO SEGURA (Para visualizacion o repartidores):");
UsuarioDTO vistaSegura = UsuarioDTO.desdeEntidad(adminKaren);
System.out.println(vistaSegura);
}
```

2. **com.utn.enums**: restringen los valores que puede tomar un atributo, asegurando la consistencia de los datos en toda la aplicación y evitando errores de tipeo manual.



- a. **Estado** → etapas de un pedido

```
1 package com.utn.enums;
2 public enum Estado { 5 usages
3     PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO 2 usages
4 }
5
```

- b. **FormaPago** → métodos disponibles

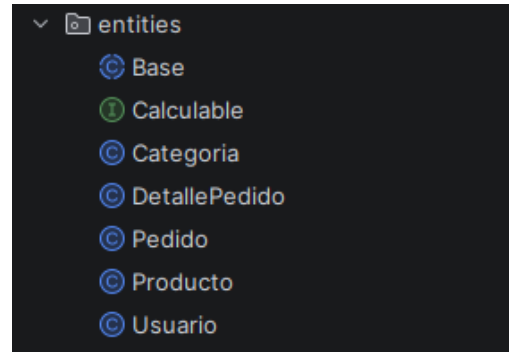
```
1 package com.utn.enums;
2 public enum FormaPago { 5 usages
3     TARJETA, TRANSFERENCIA, EFECTIVO 1 usage
4 }
```

- c. **Rol** → nivel de acceso

```
1 package com.utn.enums;
2 public enum Rol { 4 usages
3     ADMIN, USUARIO 1 usage
4 }
```

3. **com.utn.entities**: Objetos reales de la tienda →

- a. **Calculable** (Interfaz) → obliga a cualquier clase que la implemente a definir cómo calcula su total. Pedido debe tener el método correspondiente.



```
1 package com.utn.entities;
2 public interface Calculable { 1 usage 1 implementation
3     void calcularTotal(); 3 usages 1 implementation
4 }
```

- b. **Base** (clase abstracta) → como todas las entidades tiene que tener un identificador único, en lugar de escribir `private Long id`; en las 5 clases, lo heredan de Base.

```
1 package com.utn.entities;
2 > import ...
4 @Getter 5 usages 5 inheritors
5 @Setter
6 @NoArgsConstructor
7 @AllArgsConstructor
8 @SuperBuilder
9 public abstract class Base {
10     protected Long id;
11 }
```

- c. **Categoria** → Clasifica la comida. Un producto pertenece a una categoría

```
1 package com.utn.entities;
2 import lombok.*;
3 import lombok.experimental.SuperBuilder;
4 @Getter 13 usages
5 @Setter
6 @NoArgsConstructor
7 @AllArgsConstructor
8 @ToString
9 @EqualsAndHashCode(of = "nombre", callSuper = false)
10 @SuperBuilder
11 public class Categoria extends Base {
12     private String nombre;
13     private String descripcion;
14 }
```

d. **Producto** → Representa un ítem individual del menú

```
1 package com.utn.entities;
2 import lombok.*;
3 import lombok.experimental.SuperBuilder;
4 @Getter 24 usages
5 @Setter
6 @NoArgsConstructor
7 @AllArgsConstructor
8 @ToString
9 @EqualsAndHashCode(of = "nombre", callSuper = false)
10 @SuperBuilder
11 public class Producto extends Base {
12     private String nombre;
13     private Double precio;
14     private Categoria categoria;
15 }
```

e. **DetallePedido** → Línea intermedia del carrito

```
1 package com.utn.entities;
2 import lombok.*;
3 import lombok.experimental.SuperBuilder;
4 @Getter 4 usages
5 @Setter
6 @NoArgsConstructor
7 @AllArgsConstructor
8 @ToString(callSuper = true) // Le decimos que muestre el ID de la base
9 @EqualsAndHashCode(callSuper = false) // Evita que use los campos de Base para comparar detalles
10 @SuperBuilder
11 public class DetallePedido extends Base {
12     private Integer cantidad;
13     private Producto producto;
14     public Double calcularSubtotal() { 1 usage
15         // Una validación simple
16         if (this.producto == null || this.cantidad == null) {
17             return 0.0;
18         }
19         return this.cantidad * this.producto.getPrecio();
20     }
21 }
```

f. **Pedido** → Agrupa la fecha, el estado, el pago y una lista de muchos DetallePedido

```
1 package com.utn.entities;
2 import com.utn.enums.Estado;
3 import com.utn.enums.FormaPago;
4 import lombok.*;
5 import lombok.experimental.SuperBuilder;
6 import java.time.LocalDate;
7 import java.util.HashSet;
8 import java.util.Set;
9
10 @Getter 8 usages
11 @Setter
12 @NoArgsConstructor
13 @AllArgsConstructor
14 @ToString(callSuper = true) // Muestra los datos del pedido y su ID heredado
15 @EqualsAndHashCode(callSuper = false) // Evita conflictos de duplicados en colecciones por culpa de Base
16 @SuperBuilder
```



```

public class Pedido extends Base implements Calculable {
    private LocalDate fecha;
    private Estado estado;
    private FormaPago formaPago;
    private Double total; // <-- ¡FALTABA AGREGAR ESTA LÍNEA AQUÍ!

    @Builder.Default
    private Set<DetallePedido> detalles = new HashSet<>();
    public void addDetallePedido(Integer cantidad, Producto producto) { 6 usages
        if (producto != null && cantidad > 0) {
            DetallePedido detalle = DetallePedido.builder() DetallePedidoBuilder<capture of ?, capture of ?>
                .cantidad(cantidad) capture of ?
                .producto(producto)
                .build();
            detalles.add(detalle);
        }
    }

    @Override 3 usages
    public void calcularTotal() {
        double acumulador = 0.0;
        for (DetallePedido detalle : detalles) {
            acumulador += detalle.calcularSubtotal();
        }
        this.setTotal(acumulador);
    }
}

```

- g. **Usuario** → Almacena los datos del cliente o administrador y tiene un historial de sus pedidos

```

1 package com.utn.entities;
2 import com.utn.enums.Rol;
3 import lombok.*;
4 import lombok.experimental.SuperBuilder;
5 import java.util.HashSet;
6 import java.util.Set;
7
8 @Getter 9 usages
9 @Setter
10 @NoArgsConstructor
11 @AllArgsConstructor
12 @ToString
13 @EqualsAndHashCode(of = "nombre", callSuper = false)
14 @SuperBuilder
15 public class Usuario extends Base {
16     private String nombre;
17     private String apellido;
18     private String mail;
19     private String contraseña;
20     private Rol rol;
21
22     @Builder.Default
23     private Set<Pedido> pedidos = new HashSet<>();
24 }

```

4. com.utn.dtos

- a. **UsuarioDTO** → Es un record. Su fin es tomar el objeto completo de usuario, extraer sólo el nombre, el apellido y mail, y dejar afuera la contraseña y el rol

```
1 package com.utn.dtos;
2 import com.utn.entities.Usuario;
3 public record UsuarioDTO( 5 usages
4     String nombre, no usages
5     String apellido, no usages
6     String mail no usages
7 ) {
8     @ public static UsuarioDTO desdeEntidad(Usuario usuario) { 1 usage
9         return new UsuarioDTO(
10             usuario.getNombre(),
11             usuario.getApellido(),
12             usuario.getMail()
13         );
14     }
15 }
```

Relaciones entre clases:

- 5. **Herencia** → Usuario, Pedido, Producto, Categoria y DetallePedido son una extensión de Base. Comparten la estructura del identificador id.
- 6. **Asociación 1 a 1**
 - a. Un Producto tiene asignada una (1) Categoria.
 - b. Un DetallePedido hace referencia a un (1) Producto.
- 7. **Agregación/Composición 1 a muchos**
 - a. Un Pedido contiene muchos DetallePedido dentro de su lista interna. Si el pedido se cancela o elimina, conceptualmente sus detalles también pierden sentido.
 - b. Un Usuario puede acumular muchos Pedido en su historial de compras a lo largo del tiempo.

Resultado:

22:15:11: Executing ':com.utn.Main.main()'...

Task :compileJava

> Task :processResources NO-SOURCE

> Task :classes

> Task :com.utn.Main.main()

VISTA DE UN PRODUCTO INDIVIDUAL:

Producto(nombre=Hamburguesa Triple, precio=25000.0, categoria=Categoria(nombre=Hamburguesas, descripcion=Variedad de hamburguesas smash))

MENU COMPLETO DISPONIBLE EN EL FOOD STORE:

Producto(nombre=Bebida Cola, precio=1500.0, categoria=Categoria(nombre=Bebidas, descripcion=Bebidas y gaseosas lineas premium))

Producto(nombre=Pizza Mozzarella, precio=18500.0, categoria=Categoria(nombre=Pizzas, descripcion=Pizzas artesanales al horno))

Producto(nombre=Sandwich Italiano, precio=17000.0, categoria=Categoria(nombre=Sandwiches, descripcion=Sandwiches gourmet en pan baguette))

Producto(nombre=Garden Ensalada, precio=15500.0, categoria=Categoria(nombre=Ensaladas, descripcion=Opciones frescas y saludables))

Producto(nombre=Pizza Calabresa, precio=20500.0, categoria=Categoria(nombre=Pizzas, descripcion=Pizzas artesanales al horno))

Producto(nombre=Papas Fritas, precio=7500.0, categoria=Categoria(nombre=Guarniciones, descripcion=Acompañamientos crujientes))

Producto(nombre=Papas Fritas con Cheddar, precio=9000.0, categoria=Categoria(nombre=Guarniciones, descripcion=Acompañamientos crujientes))

Producto(nombre=Hamburguesa Triple, precio=25000.0, categoria=Categoria(nombre=Hamburguesas, descripcion=Variedad de hamburguesas smash))

Producto(nombre=Sandwich Milanese, precio=18000.0, categoria=Categoria(nombre=Sandwiches, descripcion=Sandwiches gourmet en pan baguette))

Producto(nombre=Hamburguesa Smash Supremo, precio=22000.0, categoria=Categoria(nombre=Hamburguesas, descripcion=Variedad de hamburguesas smash))

DETECCION DEL CLIENTE CON MAS ACTIVIDAD:

Cliente frecuente detectado: Karen Lauk

Historial de compras de este usuario:

Pedido(super=com.utn.entities.Pedido@7e11c6b1, fecha=2026-06-03, estado=CONFIRMADO, formaPago=TARJETA, total=29500.0, detalles=[DetallePedido(super=com.utn.entities.DetallePedido@27f538e7, cantidad=1, producto=Producto(nombre=Pizza Calabresa, precio=20500.0, categoria=Categoria(nombre=Pizzas, descripcion=Pizzas artesanales al horno))), DetallePedido(super=com.utn.entities.DetallePedido@99ef061a, cantidad=1,

producto=Producto(nombre=Papas Fritas con Cheddar, precio=9000.0,
categoria=Categoria(nombre=Guarniciones, descripcion=Acompañamientos crujientes))))))

-> Total leído desde el atributo del Pedido: \$29500.0

Pedido(super=com.utn.entities.Pedido@23ad4228, fecha=2026-06-03,
estado=PENDIENTE, formaPago=EFFECTIVO, total=53000.0,
detalles=[DetallePedido(super=com.utn.entities.DetallePedido@72d5f7e4, cantidad=2,
producto=Producto(nombre=Bebida Cola, precio=1500.0,
categoria=Categoria(nombre=Bebidas, descripcion=Bebidas y gaseosas lineas premium))),
DetallePedido(super=com.utn.entities.DetallePedido@6ad4108e, cantidad=2,
producto=Producto(nombre=Hamburguesa Triple, precio=25000.0,
categoria=Categoria(nombre=Hamburguesas, descripcion=Variedad de hamburguesas
smash)))]

-> Total leído desde el atributo del Pedido: \$53000.0

DETECCION DE COMIDA DUPLICADA EN LA CARTA:

❖El catalogo detecta que la hamburguesa ya existe logicamente?: true

❖El sistema permitio ingresar el elemento duplicado?: false

Cantidad de items reales remanentes en el menu: 10

VISTA DTO SEGURA (Para visualizacion o repartidores):

UsuarioDTO[nombre=Karen, apellido=Lauk, mail=karen.lauk@mail.com]

BUILD SUCCESSFUL in 790ms

2 actionable tasks: 2 executed

Consider enabling configuration cache to speed up this build:

https://docs.gradle.org/9.3.0/userguide/configuration_cache_enabling.html

22:43:57: Execution finished ':com.utn.Main.main()'

```
> Task :com.utn.Main.main()
VISTA DE UN PRODUCTO INDIVIDUAL:
Producto(nombre=Hamburguesa Triple, precio=25000.0, categoria=Categoria(nombre=Hamburguesas, descripcion=Variedad de hamburguesas smash))

MENU COMPLETO DISPONIBLE EN EL FOOD STORE:
Producto(nombre=Bebida Cola, precio=1500.0, categoria=Categoria(nombre=Bebidas, descripcion=Bebidas y gaseosas lineas premium))
Producto(nombre=Pizza Mozzarella, precio=18500.0, categoria=Categoria(nombre=Pizzas, descripcion=Pizzas artesanales al horno))
Producto(nombre=Sandwich Italiano, precio=17000.0, categoria=Categoria(nombre=Sandwiches, descripcion=Sandwiches gourmet en pan baguette))
Producto(nombre=Garden Ensalada, precio=15500.0, categoria=Categoria(nombre=Ensaladas, descripcion=Opciones frescas y saludables))
Producto(nombre=Pizza Calabresa, precio=20500.0, categoria=Categoria(nombre=Pizzas, descripcion=Pizzas artesanales al horno))
Producto(nombre=Papas Fritas, precio=7500.0, categoria=Categoria(nombre=Guarniciones, descripcion=Acompañamientos crujientes))
Producto(nombre=Papas Fritas con Cheddar, precio=9000.0, categoria=Categoria(nombre=Guarniciones, descripcion=Acompañamientos crujientes))
Producto(nombre=Hamburguesa Triple, precio=25000.0, categoria=Categoria(nombre=Hamburguesas, descripcion=Variedad de hamburguesas smash))
Producto(nombre=Sandwich Milanese, precio=18000.0, categoria=Categoria(nombre=Sandwiches, descripcion=Sandwiches gourmet en pan baguette))
Producto(nombre=Hamburguesa Smash Supremo, precio=22000.0, categoria=Categoria(nombre=Hamburguesas, descripcion=Variedad de hamburguesas smash))

DETECCION DEL CLIENTE CON MAS ACTIVIDAD:
Cliente frecuente detectado: Karen Lauk
Historial de compras de este usuario:
Pedido(super=com.utn.entities.Pedido@7e11c6b1, fecha=2026-06-03, estado=CONFIRMADO, formaPago=TARJETA, total=29500.0, detalles=[DetallePedido(super=com.utn.entities.DetallePe
-> Total leído desde el atributo del Pedido: $29500.0
Pedido(super=com.utn.entities.Pedido@23ad4228, fecha=2026-06-03, estado=PENDIENTE, formaPago=EFFECTIVO, total=53000.0, detalles=[DetallePedido(super=com.utn.entities.DetallePe
-> Total leído desde el atributo del Pedido: $53000.0

DETECCION DE COMIDA DUPLICADA EN LA CARTA:
❖El catalogo detecta que la hamburguesa ya existe logicamente?: true
❖El sistema permitio ingresar el elemento duplicado?: false
Cantidad de items reales remanentes en el menu: 10

VISTA DTO SEGURA (Para visualizacion o repartidores):
UsuarioDTO[nombre=Karen, apellido=Lauk, mail=karen.lauk@mail.com]

BUILD SUCCESSFUL in 790ms
2 actionable tasks: 2 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.3.0/userguide/configuration\_cache\_enabling.html
22:43:57: Execution finished ':com.utn.Main.main()'
```